Cache-to-Cache

Direct Semantic Communication Between Large Language Models

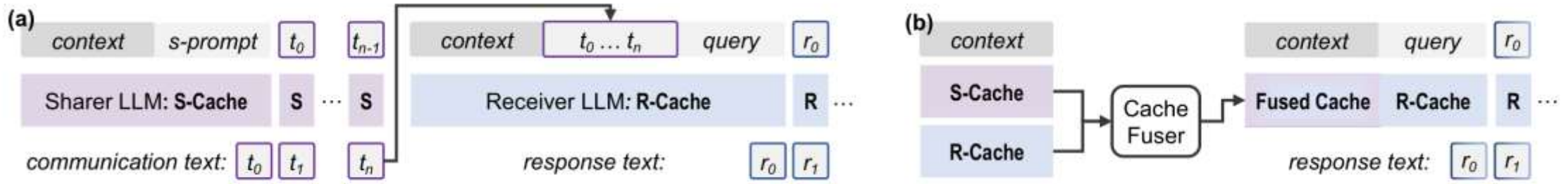




清华大学 Energy Efficient Computing Group
https://nicsefc.ee.tsinghua.edu.cn/

高效DL算法、多智能体、领域专用加速

论文 Figure 1 · 文字接口与缓存接口的计算路径



Tianyu Fu · Zihan Min · Hanling Zhang 等

清华大学 / 无问芯穹 / 香港中文大学 / 上海交通大学等 · arXiv:2510.03215v2

汇报人：刘孟玄

目录

01 问题

信息压缩
自然语言歧义
逐token中转

02 Oracle

同长度丰富化
跨模型可转换
不同层效应不同

03 方法

投影 + 融合
动态权重 + 门控
保留自身缓存

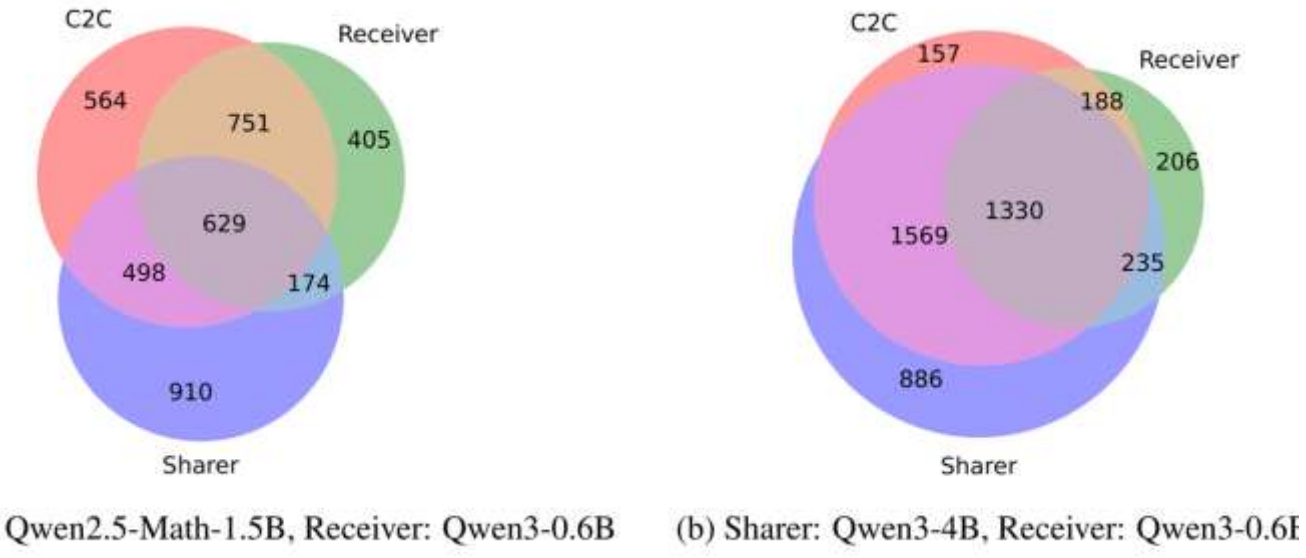
04 实验

准确率与延迟
消融与扩展
成功与失败案例

来源：论文Table1、3

模型能力互补，协作方式却共享文本接口

同一准确率 ≠ 同一答对集合



er: Qwen2.5-Math-1.5B, Receiver: Qwen3-0.6B (b) Sharer: Qwen3-4B, Receiver: Qwen3-0.6E

Figure7：相近容量互补 / 强模型向弱模型迁移；C2C-C变体。

语义通信的价值：**整合各自理解**，不只选更强模型。

C2C的研究位置

文本协作 · MetaGPT / MoA

生成分析、汇总或讨论后再回答。

路由 · RouteLLM

选择模型，通常不融合上下文理解。

缓存复用 · DroidSpeak

同架构、共享前缀；复用部分KV减少prefill。

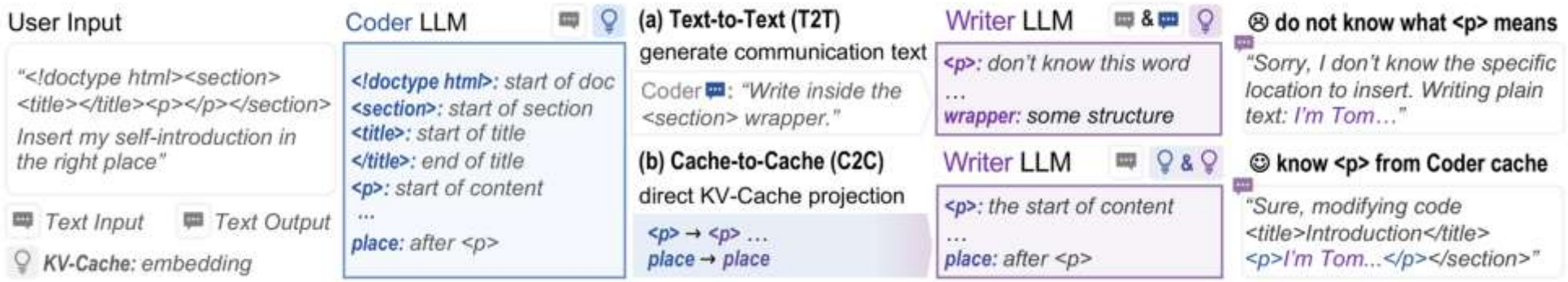
C2C · 异构语义融合

支持不同家族 / 大小 / 专长的模型。

来源：论文§2、Figure7 / Appendix A.2.3

一个Coder–Writer示例揭示三种文本瓶颈

任务：在HTML正确位置插入自我介绍；Coder知道结构，Writer负责写作。



信息瓶颈

高维理解被压成文字，
不能保留所有结构线索。

自然语言歧义

“inside wrapper”不足以
明确<p>和插入位置。

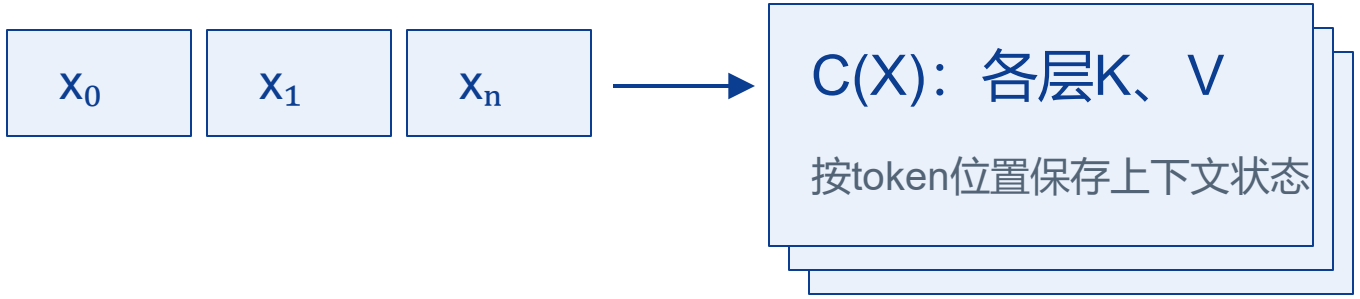
逐token中转

Sharer先生成解释，
Receiver再prefill读取。

来源：论文Figure2、§1

KV-cache连接prefill和decode，C2C修改输入侧状态

Prefill：一次处理整段输入



缓存不是权重，也不是完整可读“思维链”。

C2C替换输入侧C(X)。

Decode：逐token推进

利用输入缓存C(X)
和已生成前缀缓存C(Y)，
预测下一个token。

每一步追加新K、V；
最终回答自回归生成。

$$y_{i+1} = \mathcal{P}\left(y_i \mid C(X) \oplus C\left(Y_{[0:i]}\right)\right)$$

X为输入，Y为响应；n为输入长度，d在论文记号中是各层KV展平维度；⊕表示序列拼接。

来源：论文§3.1、Eq.1

Benefit Oracle：丢掉示例KV，增益仍然保留

方法 / 输入处理		准确率
Direct	问题 X → 保留C(X)	58.42%
Few-shot	示例 E ｜ 问题 X（全部保留）	63.39%
Oracle	~~丢弃 E~~ ｜ 仅保留被丰富的X	**62.34%**

更多token，还是更好表示?

Direct与Oracle解码时缓存长度同为 |X|。

只改变问题缓存的语义，仍提高**3.92个百分点**。

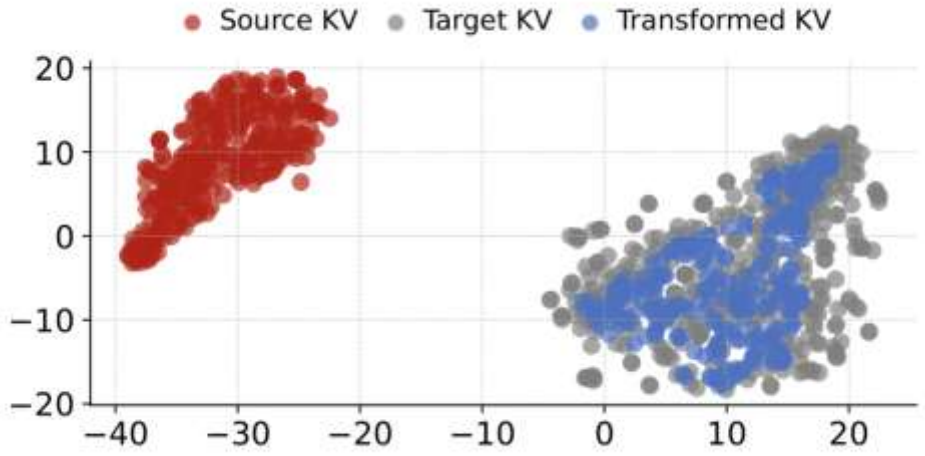
Oracle保留了示例对问题的影响。

$$C^*(X) = C_{[|E|:|E|+|X|]}(E \oplus X)$$

来源：论文§3.2 Benefit Oracle、Table1

两个诊断结论：可转换，但不能任意层直接替换

A. 表征空间可映射



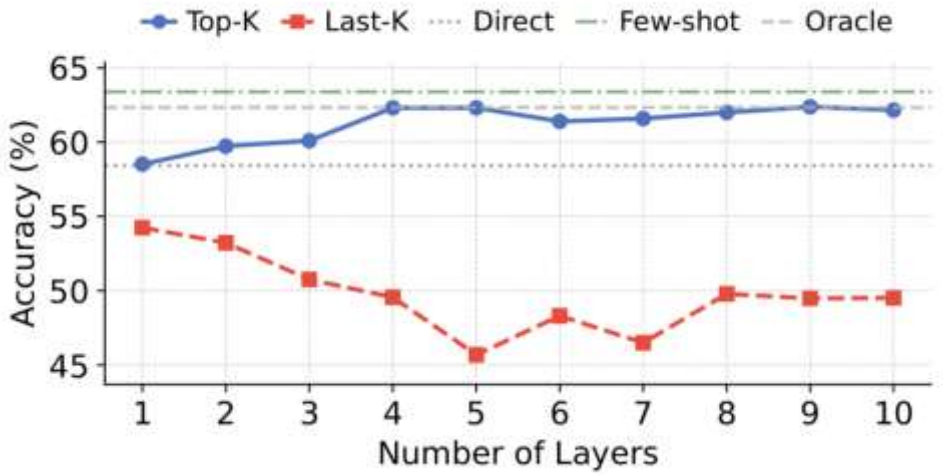
Qwen3-4B → Qwen3-0.6B，最后层KV；

3层MLP + MSE，t-SNE显示300个样本。

映射只覆盖目标子集 → **融合而非全部覆盖**。

低维重叠不等于完整语义等价。

B. 层的效果不均匀



Top-K好层优先；Last-K差层优先。

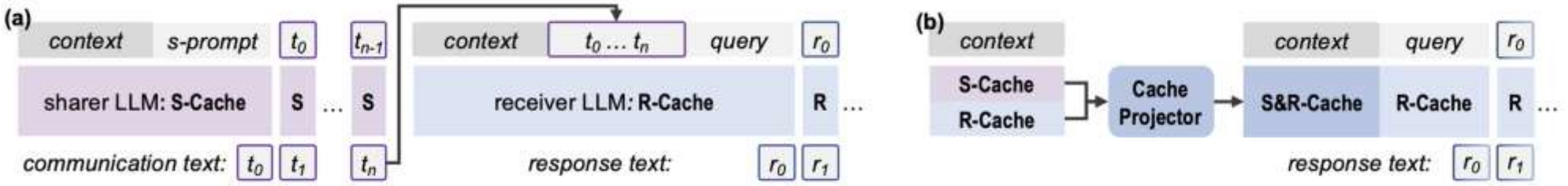
部分层注入会损伤性能 → **可学习的逐层Gate**。

Oracle MSE不是后续C2C的训练损失。

来源：论文§3.2 Transformation Oracle、Figure3 / Figure4

C2C总体流程：双prefill、缓存融合、单Receiver解码

T2T先生成通信文字；C2C在输入缓存上直接融合



① 同一context双prefill　② token / layer对齐　③ 逐层K、V融合　④ 生成回答

$$C^F = \left\{ C_n(X) + F_n\left(C_n(X), C^S_{G(n)}(X)\right) \right\}_{n=1}^{N}$$

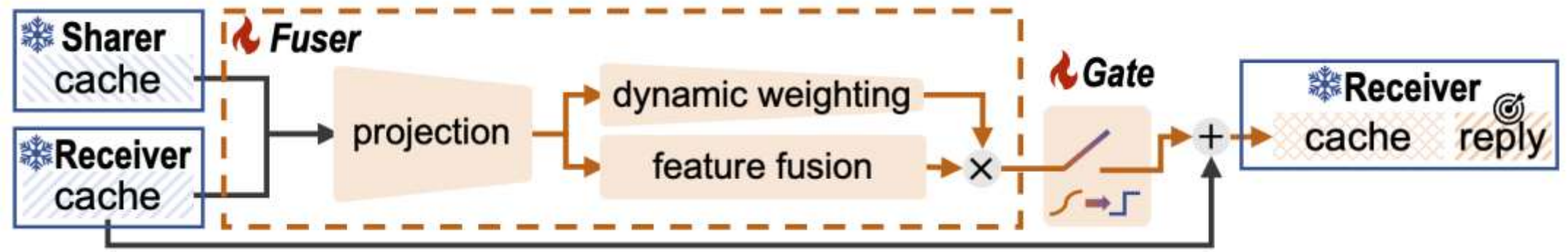
G(n)给出Sharer配对层；F_n是训练得到的通信模块，Receiver原缓存沿残差路径保留。

Sharer不decode中间文字；融合context后继续处理query / 响应。最终回答仍逐token，融合也有计算成本。

来源：Fu等作者项目页 / fuvty.github.io；论文§3.3 Eq.2

Fuser的三个部件各解决一个不同问题

作者原始架构 · 冻结缓存来源 / 可训练Fuser与Gate / Receiver残差



① 转换与特征融合

拼接两份cache并投影，
在Receiver空间产生增量；
K和V分别有对应Fuser。

② 输入相关动态权重

input-aware head modulation
根据当前输入调节各头，
控制“注入多少信息”。

③ 逐层Gate

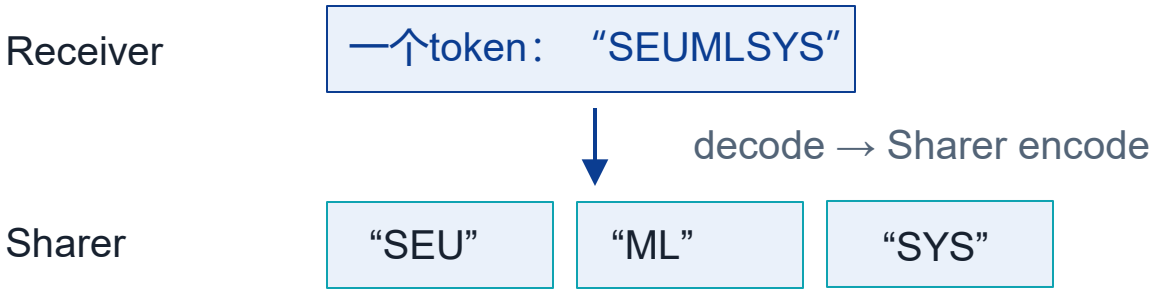
训练用Gumbel-sigmoid，
推理二值开关；
控制“哪些层注入”。

简式：C_F = C_R + g·ΔC　　**保留自身理解，再注入互补增量**；不以投影结果覆盖原cache。

来源：Fu等作者项目页 / fuvty.github.io；论文Figure5、§3.3（简式仅用于讲解）

跨模型对齐：token对应与末层优先配对

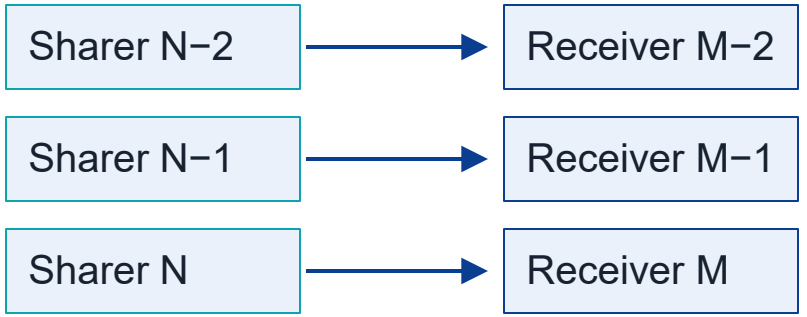
Token：回到字符串，再匹配覆盖



一对多时选最大字符串覆盖位置；

Layer：先匹配最末层

层数示意；从末层逐层向前配对



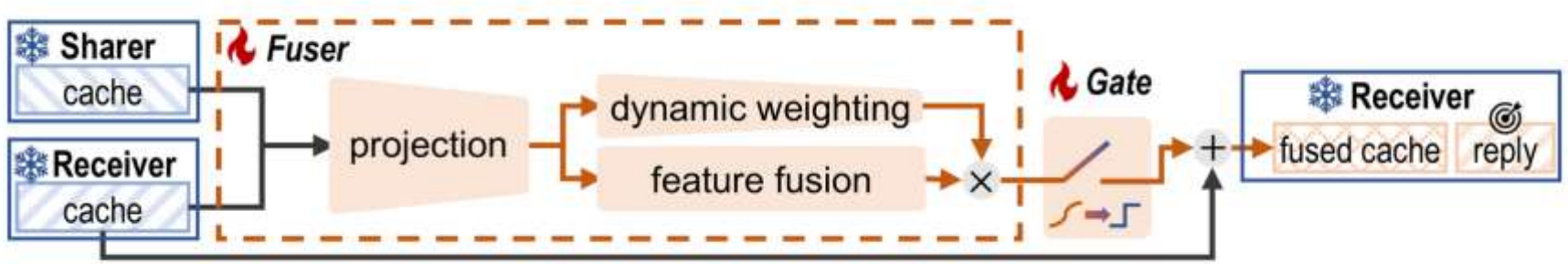
到较浅模型第一层停止

两个启发式的对齐规则

来源：论文Appendix A.2.1 Cache Alignment

冻结两模型，用响应预测损失学习通信模块

双模型forward取得缓存 → 融合 → Receiver响应预测 → 只更新Fuser与Gate



$$\mathcal{L} = -\sum_{t} \log p_\theta \left(y_t \mid X, y_{<t}; F \right)$$

OpenHermes2.5前500k样本，1 epoch / 1929 steps

macro batch 256 · max sequence 2048 · lr 1e−4

warmup 10% · seed 42 · gate温度 1 → 0.001

来源：论文Figure5、§4.1、Appendix A.2.2

实验设置

四个数据集

MMLU-Redux：多学科

OpenBookQA：科学

ARC-C：推理挑战集

C-Eval：中文知识

五种方法

Receiver · Sharer · Routing

T2T文本协作 · C2C缓存融合

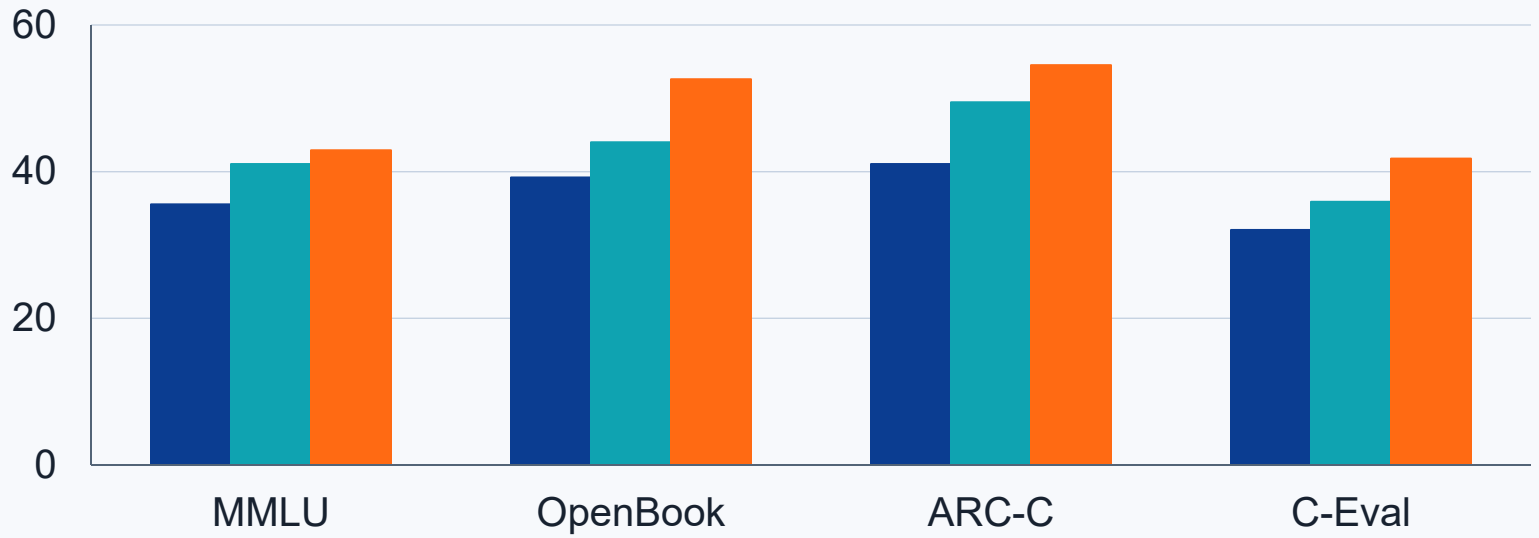
R：Qwen3-0.6B；主Sharer：

Qwen2.5-0.5B / Llama3.2-1B /

Qwen3-4B-Base

来源：论文§4.1、Appendix A.1 / A.2

四任务均提高



准确率（%），柱图从0开始

Receiver T2T C2C

相对T2T的百分点增益

MMLU +1.89 / ARC +5.04

OpenBook +8.60

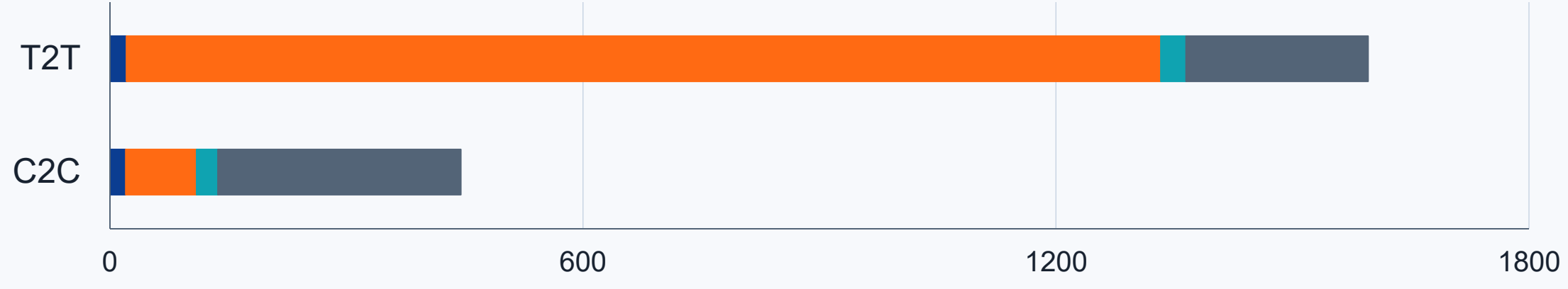
C-Eval +5.89

方法	MMLU	OpenBook	ARC-C	C-Eval
Receiver	35.53	39.20	41.04	32.04
Sharer	38.42	45.60	42.09	40.21
Routing	35.58	40.80	40.70	34.61
T2T	41.03	44.00	49.48	35.88
C2C	**42.92**	**52.60**	**54.52**	**41.77**

● 最高

来源：论文Table4

时间分解：1312ms中间解码被90ms融合取代



1596 → 445 ms（3.59×）

路径	S prefill	通信	R prefill	R decode	输出tokens (S / R)
T2T →	21 ms	1312 ms	32 ms	231 ms	80 / 10
C2C →	20 ms	90 ms	27 ms	308 ms	0 / 12

输入tokens：T2T S/R=103/332，C2C=170/170

来源：论文Table3

不同家族与专长也可通信，方向交换仍有收益

Sharer → Receiver（B）	R单独	S单独	T2T	C2C	
Gemma3-1 → Qwen3-0.6	35.53	31.75	41.35	**45.90**	●
Qwen2.5-Math-1.5 → Qwen3-0.6	35.53	39.86	43.71	**46.13**	●
Qwen2.5-Coder-0.5 → Qwen3-0.6	35.53	25.09	39.74	**46.89**	●
Qwen3-0.6 → Qwen2.5-0.5（交换）	38.42	35.53	32.12	**43.47**	
Qwen2.5-0.5 → Qwen3-0.6（常规）	35.53	38.42	41.03	**46.50**	

家族、专长和角色方向不同，C2C仍有收益；

每个模型对都训练相应通信模块。

来源：论文Table7、Appendix A.4.3

消融：保留自身缓存，比纯投影覆盖重要得多

结构 / 模块	MMLU	ARC-C	OpenBook	C-Eval	平均
S → Project → 替换R	20.01	19.57	21.80	21.41	20.70
R + Fuse(R,S)	43.36	51.65	47.60	36.91	44.88
R + Gate·Fuse(R,S)	42.92	54.52	52.60	41.77	**47.95** ●

平均：融合 +24.18个百分点，Gate再 +3.07；但Gate在MMLU上下降0.44。

替代解释对照	可训练参数	MMLU
Single（R微调）	596M	36.81
Identical（同模型）	529M	42.17
C2C（异构）	478M	42.92 ●

这个实验是否公平我认为还有待商榷
会在后面详细讨论

来源：论文Table8（accuracy %，平均按表列）与Table6

模型规模与序列长度扩展：收益随条件变化

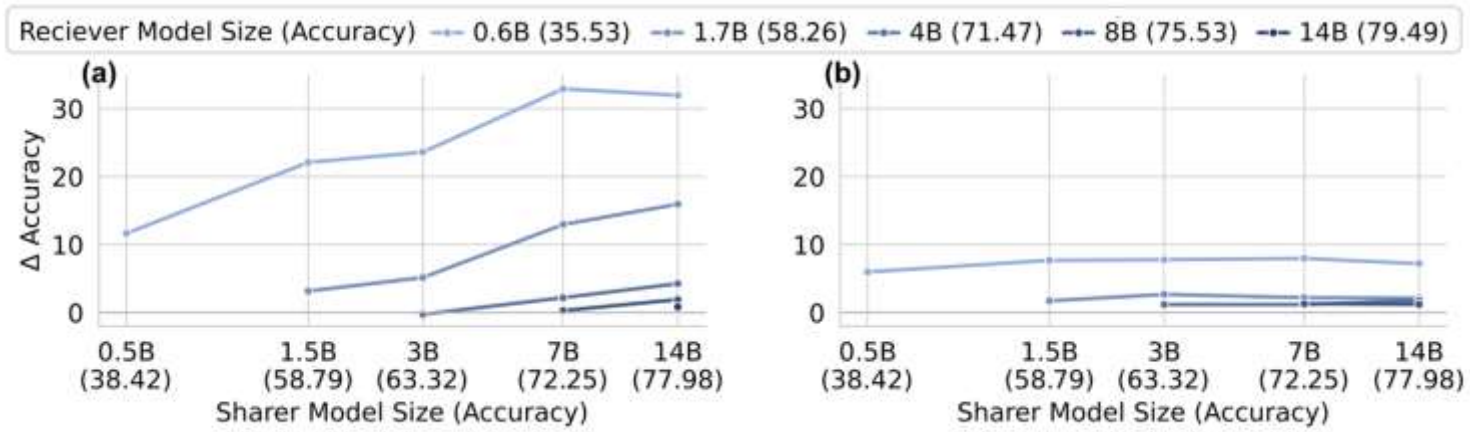


Figure6：Receiver 0.6–14B / Sharer 0.5–14B；MMLU auxiliary训练

规模曲线

强Sharer可提供更多知识；

Receiver越强，相对收益可能越小。

弱Sharer可能带来噪声。

并非容量越大就必然更好。

输入区间	Receiver	Sharer	T2T	C2C
0–4k	30.52	24.94	33.46	37.31
4–8k	26.03	23.18	29.70	34.01
8k+	25.99	16.44	25.64	30.72

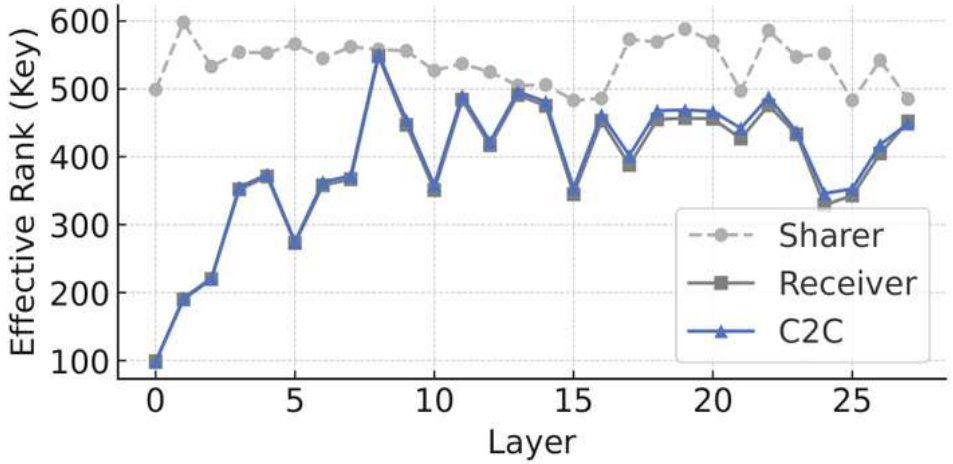
LongBench-E综合分

独立3/4训练、1/4评估；

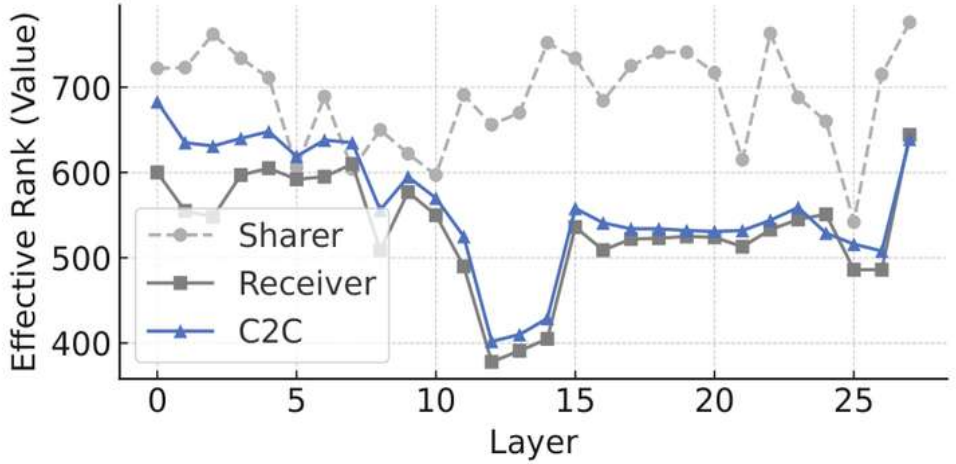
Qwen0.5B → Qwen0.6B，

来源：论文Figure6、Table5

行为分析：表征更丰富，Gate稀疏性受训练影响



(a) Effective rank of the Key cache.



(b) Effective rank of the Value cache.

Figure12：各层Key / Value effective rank变化

平均有效秩 / Table2

Sharer / Receiver / C2C

K：539 / 388 / 395

V：689 / 532 / 560

Value浅层丰富化较明显，
Key变化更小；秩≠知识正确。

来源：论文Table2、Figure12、Appendix A.4.1 / A.4.2

成功与失败案例：协作改变理解，也会传递偏差

成功：Hill的道德评价问题

关键：评价行为，还是行为者？

Receiver → A	Sharer → A
T2T → C	**C2C → B（正确）**

融合后焦点从“行为对错”转向“行为者的态度与品格”。

失败：现金制转权责发生制

赊销5600，已收1900，应计工资1200。

正确增加额：5600−1900−1200 = 2500（A）

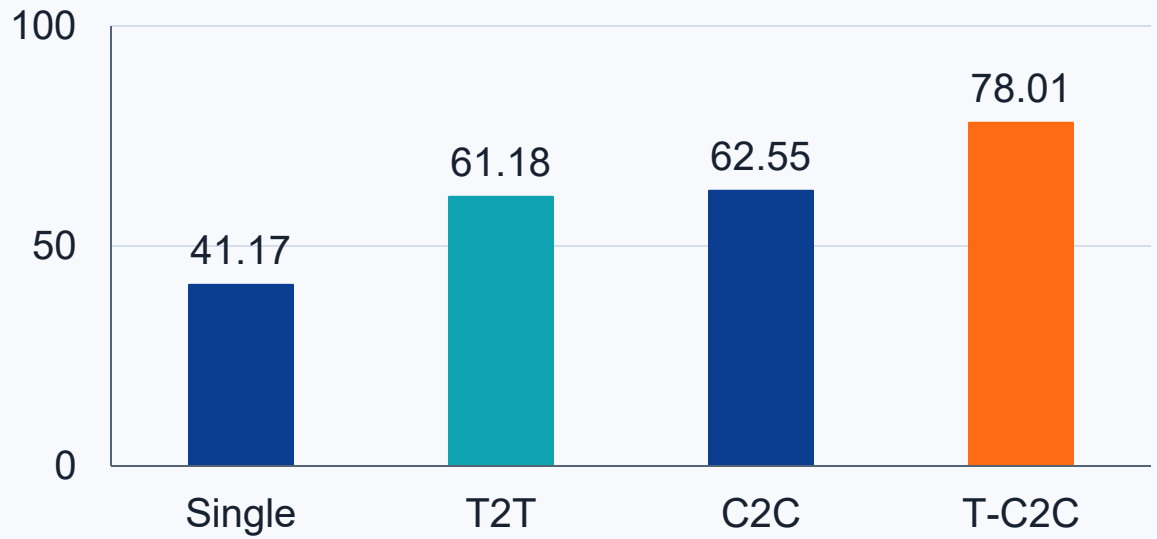
Receiver选A，但原解释写2400。

Sharer / T2T / C2C选B：3700，遗漏工资费用1200。

“选对答案”不能改写为“推理正确”。

来源：论文Appendix A.4.4 / A.4.6，Text5–14

从双模型走向多Sharer与文本/缓存混合协作



文本与缓存可以互补

解释器分析问题，求解器计算答案。

T-C2C保留解释文本，并融合问题与解释KV；

多源缓存 → 一个Receiver

Receiver单独	35.53
4B-Base → R	60.71
Math1.5B → R	46.13
双Sharer → R	64.60

两个模块分别训练后组合（Table13）。

共享latent：M个投影 + N个融合

两Receiver双源：51.08 / 59.81；

4B单源：51.23 / 59.17（Table14）。

来源：论文Table13–15、Appendix A.5

阅读总结与评价

论文总结

① 不同模型的KV Cache可以相互转换

② 传统 Text2Text 会出现语义的损失通过Cache2Cache可以在一定程度上丰富模型的知识储备

③ 通过Cache2Cache可以很大程度省去分享者模型中间逐token解码的时间

可能存在的问题与启发

① 实验的场景设置比较 Toy，并不是真正的 Agent 协作场景，真正的Agent场景更多的是主agent委派任务给自agent，论文的场景是主agent和子agent收到同样的提示词，更像是某种知识增强——能否在真正的多agent协作场景下实现通过Cache传递信息?

② 这个通信模块是个很大的神经网络，足足0.48B，而主实验用的模型也才0.5-0.6B，不太好说到底是学会了怎么转换模型之间的Cache还是把训练集的知识学进这个转换器里面了（前文的实验对比了可学习参数）——一定要训练模型吗？有没有可能有什么Train-Free的方法实现Cache的转换（如果不同模型不容易实现的话，相同模型是否可行？——似乎已有一些相关工作）？

依据：论文Limitations、Appendix A.4.6 / A.5